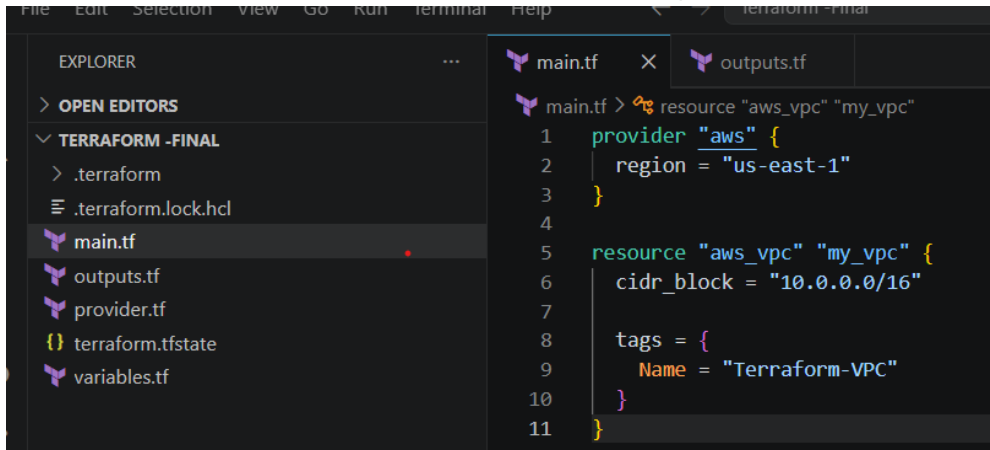


Terraform Final

1) Create VPC

- Create a folder and create main.tf file and written Script

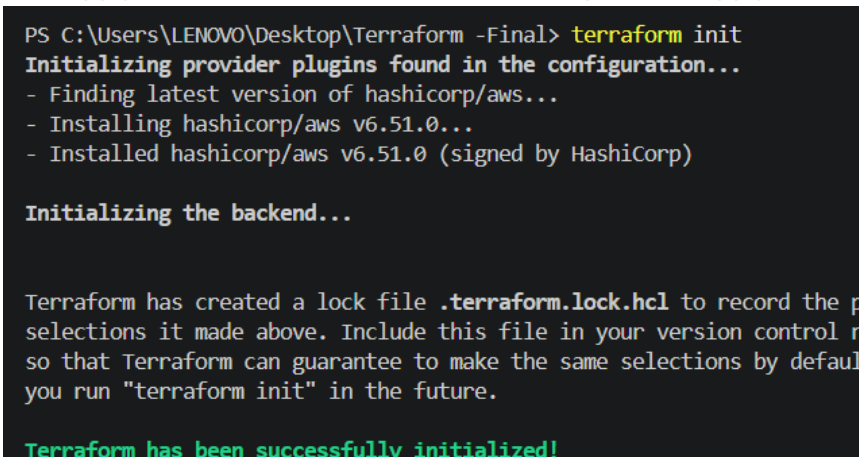


```
File Edit Selection View Go Run Terminal Help
Terraform - Final

EXPLORER
> OPEN EDITORS
TERRAFORM -FINAL
> .terraform
.terraform.lock.hcl
main.tf
outputs.tf
provider.tf
terraform.tfstate
variables.tf

main.tf
1 resource "aws_vpc" "my_vpc"
2   provider "aws" {
3     region = "us-east-1"
4   }
5   resource "aws_vpc" "my_vpc" {
6     cidr_block = "10.0.0.0/16"
7   }
8   tags = {
9     Name = "Terraform-VPC"
10  }
11 }
```

- Now apply terraform commands init, validate, plan and apply

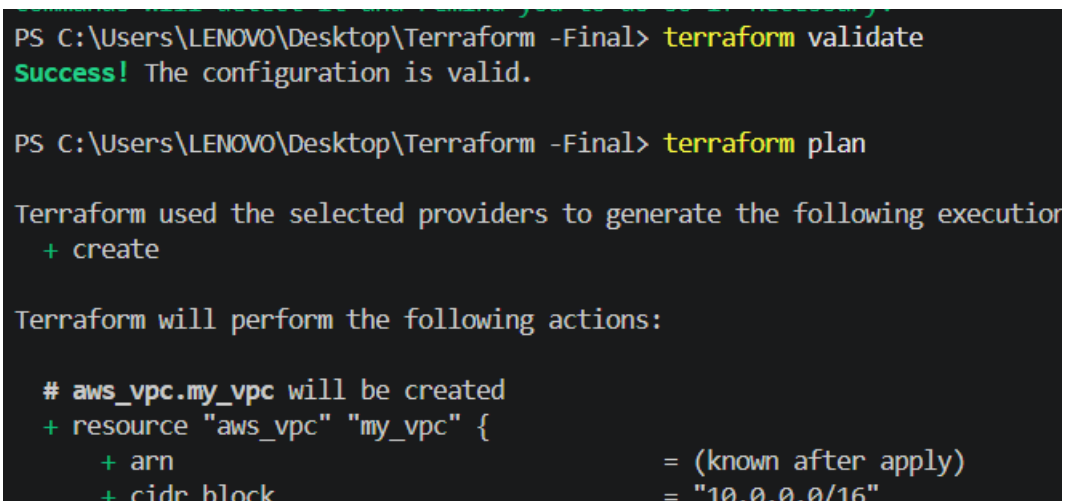


```
PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.51.0...
- Installed hashicorp/aws v6.51.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the p
selections it made above. Include this file in your version control r
so that Terraform can guarantee to make the same selections by default
you run "terraform init" in the future.

Terraform has been successfully initialized!
```



```
PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform validate
Success! The configuration is valid.

PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform plan

Terraform used the selected providers to generate the following execution
+ create

Terraform will perform the following actions:

# aws_vpc.my_vpc will be created
+ resource "aws_vpc" "my_vpc" {
+   arn                                = (known after apply)
+   cidr_block                         = "10.0.0.0/16"
```

Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee that the next apply will be identical to the plan above.
PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform apply

Terraform used the selected providers to generate the following execution plan. Resources to be created:

Terraform will perform the following actions:

```
# aws_vpc.my_vpc will be created
+ resource "aws_vpc" "my_vpc" {
```

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_vpc.my_vpc: Creating...

aws_vpc.my_vpc: Creation complete after 5s [id=vpc-075aae91705e8cdd3]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

- Now check in AWS Console Vpc has been created

The screenshot shows the AWS Management Console interface. On the left, the 'VPC dashboard' is visible with a sidebar containing 'Your VPCs', 'Subnets', 'Route tables', and 'Internet gateways'. The main content area displays the details for the VPC 'vpc-075aae91705e8cdd3 / Terraform-VPC'. The 'Details' tab is selected, showing the following information:

Details	State	Block Public Access
VPC ID vpc-075aae91705e8cdd3	State Available	Block Public Access Off
DNS resolution Enabled	Tenancy default	DHCP option set dopt-0c4b0af3221994469
Main network ACL acl-0cf7ec45a108559c7	Default VPC No	IPv4 CIDR 10.0.0.0/16

2) Create Internet gateway

- Now on the same main.tf written script for IGW

The screenshot shows a code editor with two tabs: 'main.tf' and 'outputs.tf'. The 'main.tf' tab is active, displaying the following Terraform configuration for an Internet Gateway:

```
11 resource "aws_internet_gateway" "my_igw"
12 {
13   resource "aws_internet_gateway" "my_igw" {
14     vpc_id = aws_vpc.my_vpc.id
15
16     tags = {
17       Name = "Terraform-IGW"
18     }
19   }
```

- Now do terraform commands plan and apply

```
PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform plan
aws_vpc.my_vpc: Refreshing state... [id=vpc-075aae91705e8cdd3]

Terraform used the selected providers to generate the following execution plan:
+ create

Terraform will perform the following actions:

# aws_internet_gateway.my_igw will be created
+ resource "aws_internet_gateway" "my_igw" {
```

```
Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee
PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform apply -auto-approve
aws_vpc.my_vpc: Refreshing state... [id=vpc-075aae91705e8cdd3]

Terraform used the selected providers to generate the following execution plan:
+ create

Terraform will perform the following actions:

# aws_internet_gateway.my_igw will be created
```

```
Plan: 1 to add, 0 to change, 0 to destroy.
aws_internet_gateway.my_igw: Creating...
aws_internet_gateway.my_igw: Creation complete after 4s [id=igw-0a0b82beebfc6267f]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\LENOVO\Desktop\Terraform -Final> █
```

- Now check for above VPC IGW attached or not

The screenshot shows the AWS Management Console interface for an Internet Gateway. The breadcrumb navigation at the top indicates the path: VPC > Internet gateways > igw-0a0b82beebfc6267f. The main header displays the gateway's name: igw-0a0b82beebfc6267f / Terraform-IGW.

Details:

- Internet gateway ID:** igw-0a0b82beebfc6267f
- State:** Attached (indicated by a green checkmark icon)
- VPC ID:** vpc-075aae91705e8cdd3 | Terraform-VPC
- Owner:** 928430096503

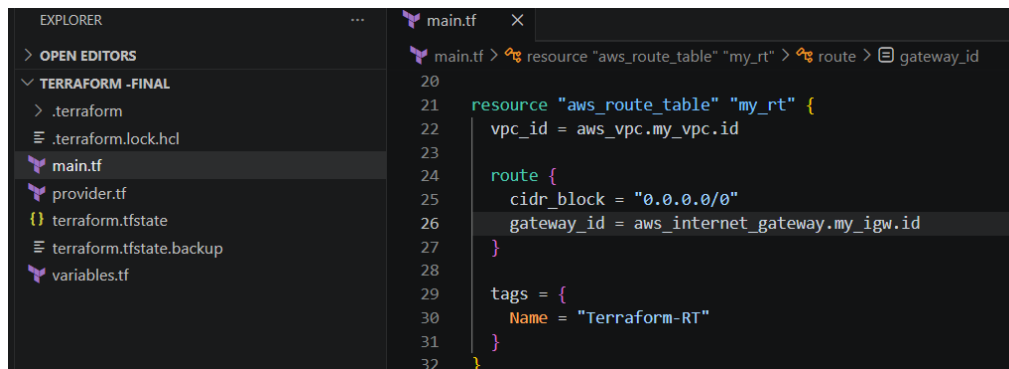
Tags (1):

Key	Value
Name	Terraform-IGW

The left sidebar shows the 'VPC dashboard' with a filter by VPC set to 'vpc-075aae91705e8cdd3'. Under the 'Virtual private cloud' section, 'Internet gateways' is selected, showing one gateway.

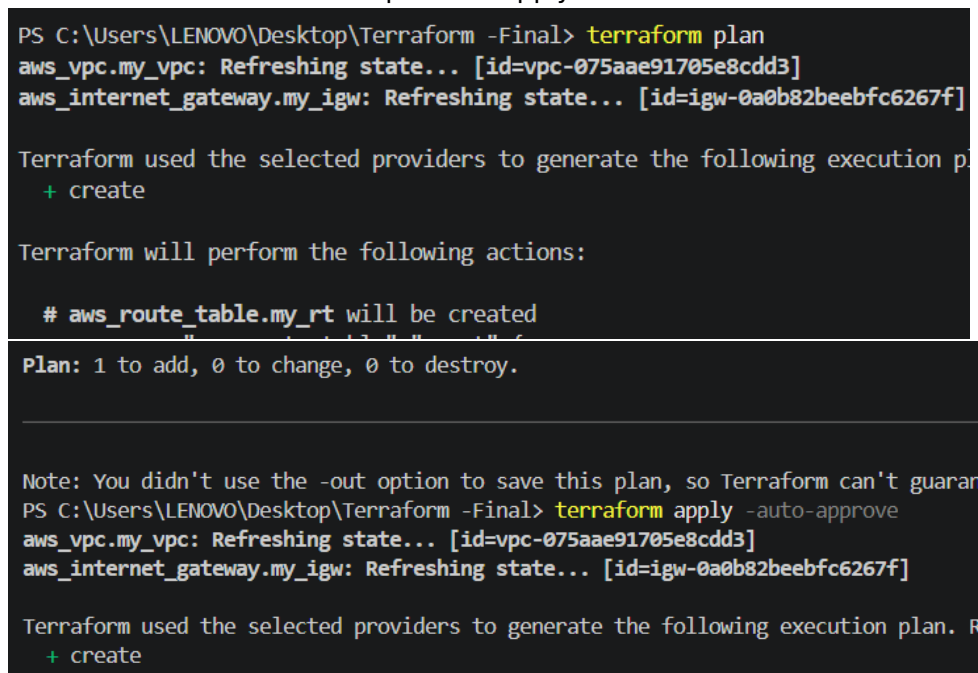
3) Create Custom Route Table

- Create Custom Route Table added this resource below IGW resource main.tf file



```
20
21 resource "aws_route_table" "my_rt" {
22   vpc_id = aws_vpc.my_vpc.id
23
24   route {
25     cidr_block = "0.0.0.0/0"
26     gateway_id = aws_internet_gateway.my_igw.id
27   }
28
29   tags = {
30     Name = "Terraform-RT"
31   }
32 }
```

- Now do terraform commands plan and apply



```
PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform plan
aws_vpc.my_vpc: Refreshing state... [id=vpc-075aae91705e8cdd3]
aws_internet_gateway.my_igw: Refreshing state... [id=igw-0a0b82beebfc6267f]

Terraform used the selected providers to generate the following execution plan:
+ create

Terraform will perform the following actions:

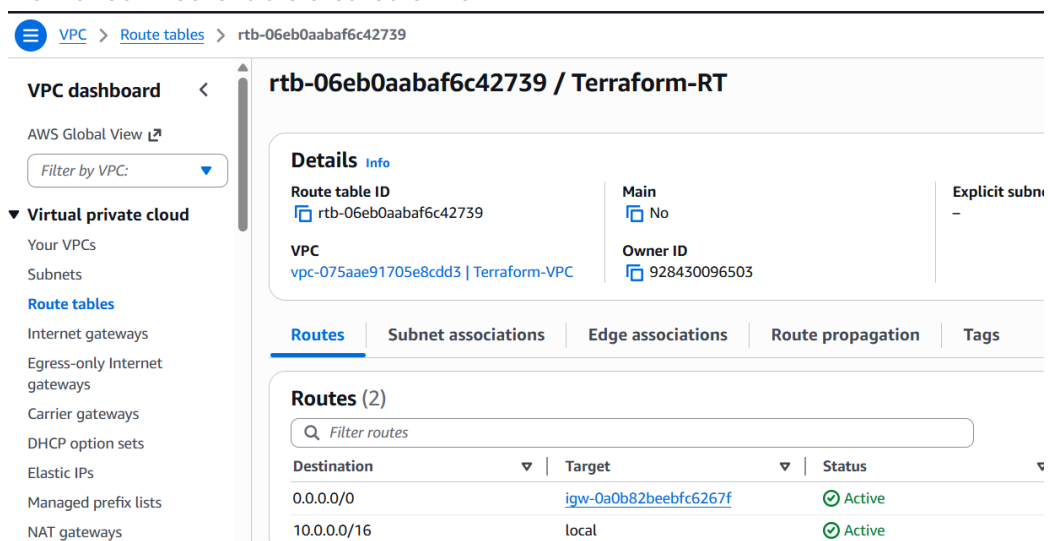
# aws_route_table.my_rt will be created
+ aws_route_table.my_rt: creating [id=rtb-06eb0aabaf6c42739]

Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee
PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform apply -auto-approve
aws_vpc.my_vpc: Refreshing state... [id=vpc-075aae91705e8cdd3]
aws_internet_gateway.my_igw: Refreshing state... [id=igw-0a0b82beebfc6267f]

Terraform used the selected providers to generate the following execution plan. R
+ create
```

- Now check Route table created or not



The screenshot shows the AWS Management Console interface for a Route Table. The breadcrumb navigation at the top indicates the path: VPC > Route tables > rtb-06eb0aabaf6c42739. The main heading is "rtb-06eb0aabaf6c42739 / Terraform-RT".

Details

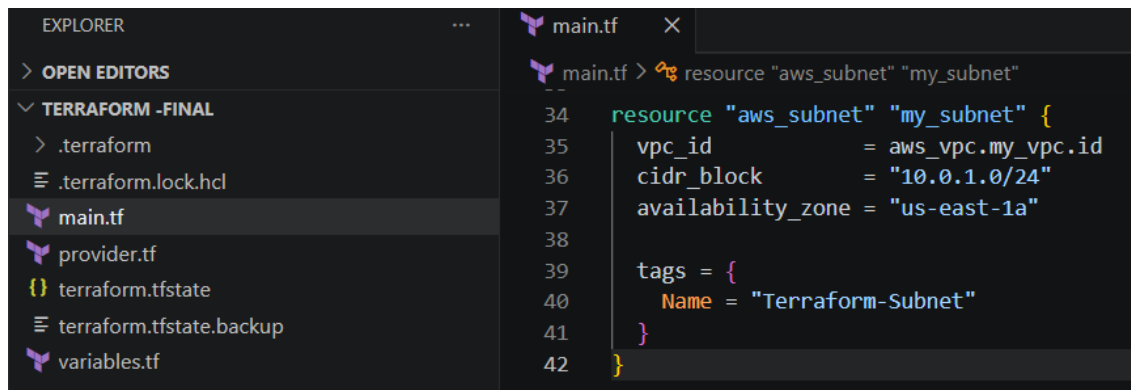
Route table ID	rtb-06eb0aabaf6c42739	Main	No	Explicit subnet	-
VPC	vpc-075aae91705e8cdd3 Terraform-VPC	Owner ID	928430096503		

Routes (2)

Destination	Target	Status
0.0.0.0/0	igw-0a0b82beebfc6267f	Active
10.0.0.0/16	local	Active

4) Create Subnet

- Create Subnet add this resource below the Route Table resource in main.tf file

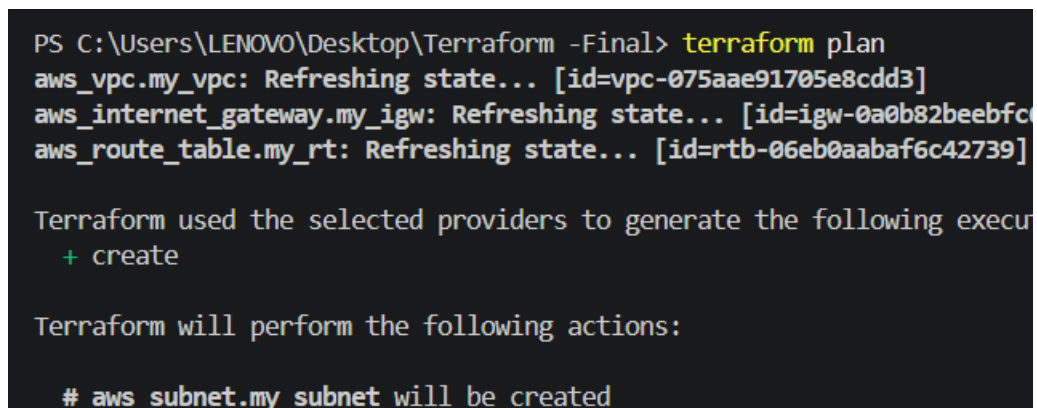


```
EXPLORER
> OPEN EDITORS
TERRAFORM -FINAL
> .terraform
.terraform.lock.hcl
main.tf
provider.tf
terraform.tfstate
terraform.tfstate.backup
variables.tf

main.tf
resource "aws_subnet" "my_subnet"

34 resource "aws_subnet" "my_subnet" {
35     vpc_id          = aws_vpc.my_vpc.id
36     cidr_block      = "10.0.1.0/24"
37     availability_zone = "us-east-1a"
38
39     tags = {
40         Name = "Terraform-Subnet"
41     }
42 }
```

- Now do terraform commands plan and apply

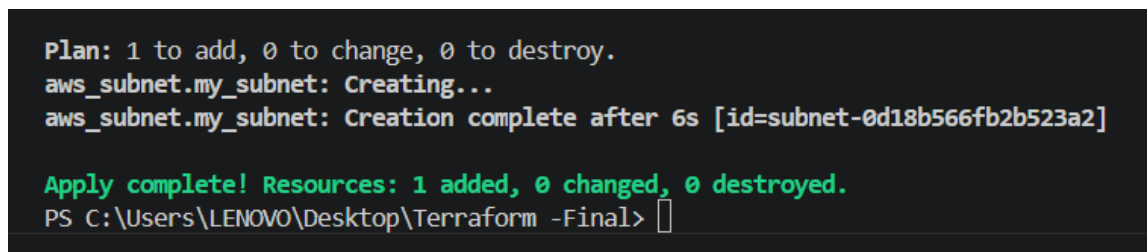


```
PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform plan
aws_vpc.my_vpc: Refreshing state... [id=vpc-075aae91705e8cdd3]
aws_internet_gateway.my_igw: Refreshing state... [id=igw-0a0b82beebfc]
aws_route_table.my_rt: Refreshing state... [id=rtb-06eb0aabaf6c42739]

Terraform used the selected providers to generate the following execution
+ create

Terraform will perform the following actions:

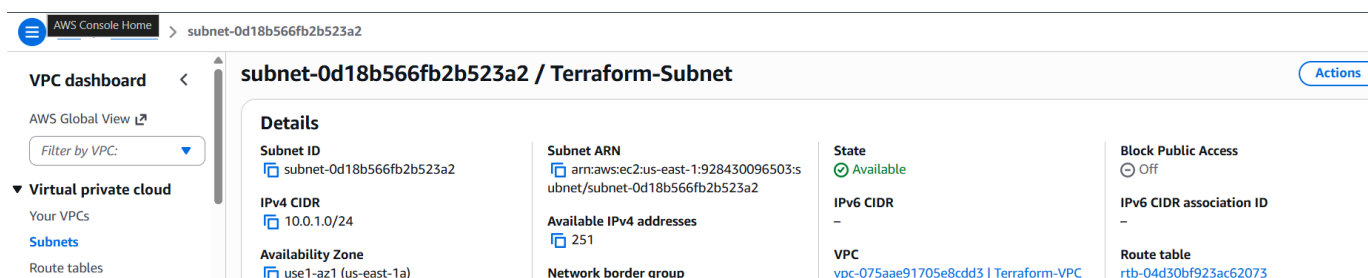
# aws_subnet.my_subnet will be created
```



```
Plan: 1 to add, 0 to change, 0 to destroy.
aws_subnet.my_subnet: Creating...
aws_subnet.my_subnet: Creation complete after 6s [id=subnet-0d18b566fb2b523a2]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\LENOVO\Desktop\Terraform -Final>
```

- Now check in subnets created or not

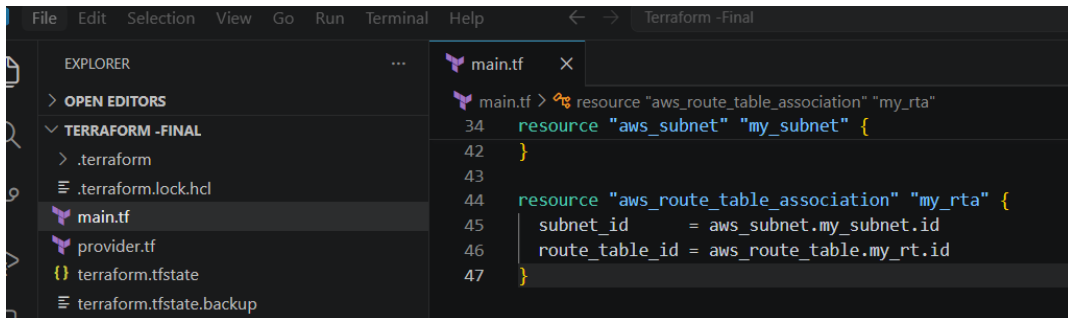


The screenshot shows the AWS Management Console interface. On the left, the 'VPC dashboard' is visible with a sidebar containing 'Virtual private cloud', 'Your VPCs', 'Subnets', and 'Route tables'. The main content area is titled 'subnet-0d18b566fb2b523a2 / Terraform-Subnet'. It displays various details for the subnet:

- Subnet ID:** subnet-0d18b566fb2b523a2
- Subnet ARN:** arn:aws:ec2:us-east-1:928430096503:subnet/subnet-0d18b566fb2b523a2
- State:** Available
- Block Public Access:** Off
- IPv4 CIDR:** 10.0.1.0/24
- Available IPv4 addresses:** 251
- Network border group:** (empty)
- IPv6 CIDR:** -
- VPC:** vpc-075aae91705e8cdd3 | Terraform-VPC
- Route table:** rtb-04d30bf923ac62073

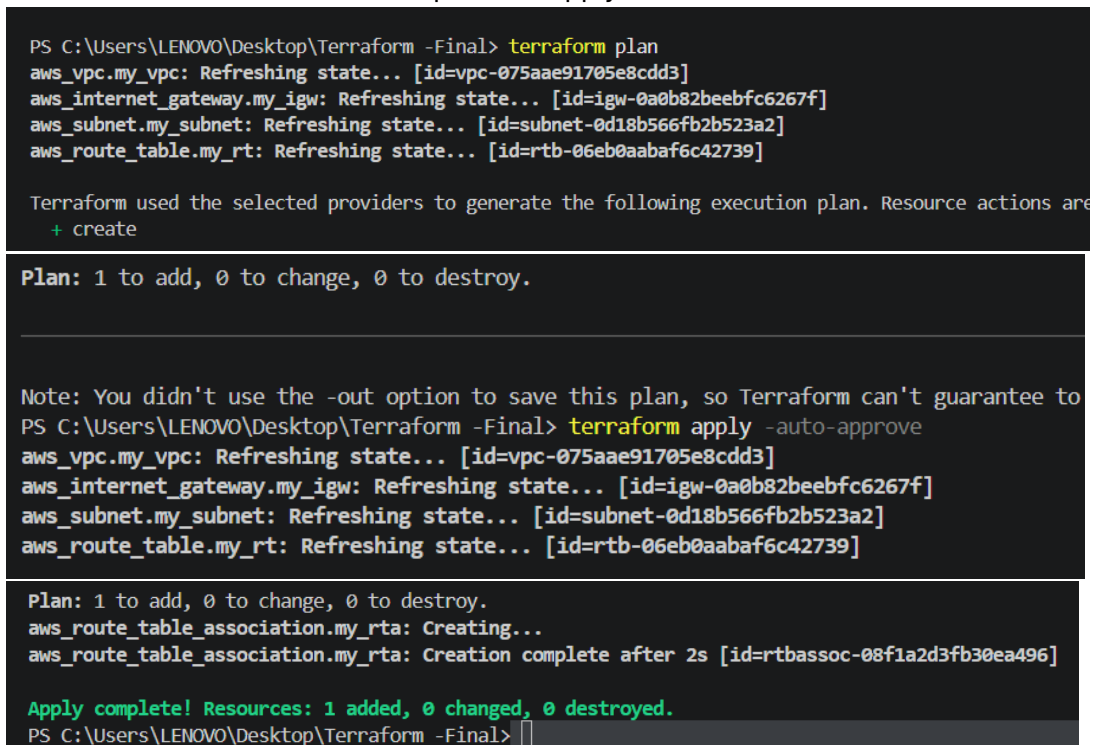
5) Associate subnet with Route Table

- Add below resource for subnet with route table in main.tf file



```
File Edit Selection View Go Run Terminal Help
EXPLORER
  OPEN EDITORS
  TERRAFORM -FINAL
    .terraform
    .terraform.lock.hcl
    main.tf
    provider.tf
    terraform.tfstate
    terraform.tfstate.backup
main.tf
  main.tf > resource "aws_route_table_association" "my_rta"
  34 resource "aws_subnet" "my_subnet" {
  42 }
  43
  44 resource "aws_route_table_association" "my_rta" {
  45     subnet_id = aws_subnet.my_subnet.id
  46     route_table_id = aws_route_table.my_rt.id
  47 }
```

- Now do terraform commands like plan and apply



```
PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform plan
aws_vpc.my_vpc: Refreshing state... [id=vpc-075aae91705e8cdd3]
aws_internet_gateway.my_igw: Refreshing state... [id=igw-0a0b82beebfc6267f]
aws_subnet.my_subnet: Refreshing state... [id=subnet-0d18b566fb2b523a2]
aws_route_table.my_rt: Refreshing state... [id=rtb-06eb0aabaf6c42739]

Terraform used the selected providers to generate the following execution plan. Resource actions are
+ create

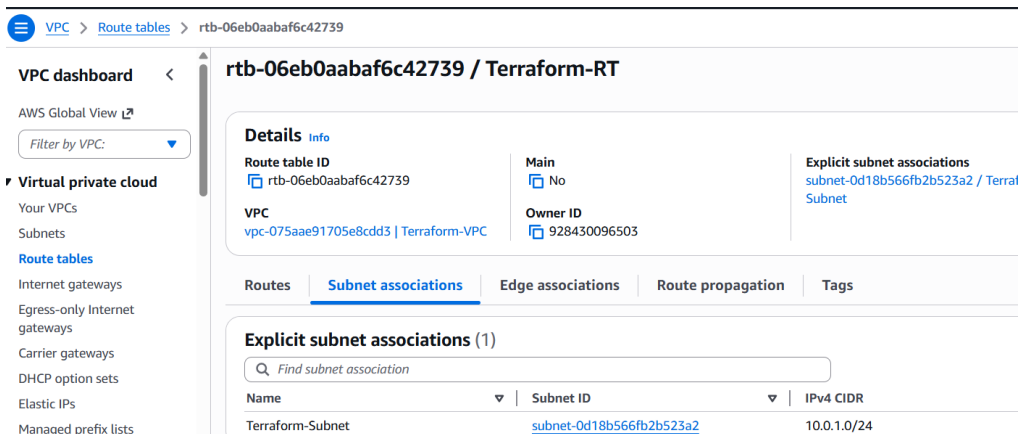
Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to
PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform apply -auto-approve
aws_vpc.my_vpc: Refreshing state... [id=vpc-075aae91705e8cdd3]
aws_internet_gateway.my_igw: Refreshing state... [id=igw-0a0b82beebfc6267f]
aws_subnet.my_subnet: Refreshing state... [id=subnet-0d18b566fb2b523a2]
aws_route_table.my_rt: Refreshing state... [id=rtb-06eb0aabaf6c42739]

Plan: 1 to add, 0 to change, 0 to destroy.
aws_route_table_association.my_rta: Creating...
aws_route_table_association.my_rta: Creation complete after 2s [id=rtbassoc-08f1a2d3fb30ea496]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\LENOVO\Desktop\Terraform -Final>
```

- Now check in Route table subnet associations attached



VPC > Route tables > rtb-06eb0aabaf6c42739

rtb-06eb0aabaf6c42739 / Terraform-RT

Details Info

Route table ID	rtb-06eb0aabaf6c42739	Main	No	Explicit subnet associations subnet-0d18b566fb2b523a2 / Terraform-Subnet
VPC	vpc-075aae91705e8cdd3 Terraform-VPC	Owner ID	928430096503	

Routes Subnet associations Edge associations Route propagation Tags

Explicit subnet associations (1)

Find subnet association

Name	Subnet ID	IPv4 CIDR
Terraform-Subnet	subnet-0d18b566fb2b523a2	10.0.1.0/24

6) Create Security Group to allow port 22,80,443

- Create Security Groups in main.tf file

```
main.tf  X
main.tf > resource "aws_security_group" "web_sg"
40
49 resource "aws_security_group" "web_sg" {
50     name           = "Terraform-Web-SG"
51     description    = "Allow SSH HTTP HTTPS"
52     vpc_id         = aws_vpc.my_vpc.id
53
54     ingress {
55         description = "SSH"
56         from_port   = 22
57         to_port     = 22
58         protocol    = "tcp"
59         cidr_blocks = ["0.0.0.0/0"]
60     }
61
62     ingress {
63         description = "HTTP"
64         from_port   = 80
65         to_port     = 80
66         protocol    = "tcp"
67         cidr_blocks = ["0.0.0.0/0"]
68     }
```

```
69
70     ingress {
71         description = "HTTPS"
72         from_port   = 443
73         to_port     = 443
74         protocol    = "tcp"
75         cidr_blocks = ["0.0.0.0/0"]
76     }
77
78     egress {
79         from_port   = 0
80         to_port     = 0
81         protocol    = "-1"
82         cidr_blocks = ["0.0.0.0/0"]
83     }
84
85     tags = {
86         Name = "Terraform-Web-SG"
87     }
88 }
```

- Now do terraform commands plan and apply

```
PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform plan
Acquiring state lock. This may take a few moments...
aws_vpc.my_vpc: Refreshing state... [id=vpc-075aae91705e8cdd3]
aws_internet_gateway.my_igw: Refreshing state... [id=igw-0a0b82beebfc6267f]
aws_subnet.my_subnet: Refreshing state... [id=subnet-0d18b566fb2b523a2]
aws_route_table.my_rt: Refreshing state... [id=rtb-06eb0aabaf6c42739]
aws_route_table_association.my_rta: Refreshing state... [id=rtbassoc-08f1a2d3fb30ea496]

Terraform used the selected providers to generate the following execution plan. Resources to be
+ create
```

Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to ta

```
PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform apply -auto-approve
aws_vpc.my_vpc: Refreshing state... [id=vpc-075aae91705e8cdd3]
aws_internet_gateway.my_igw: Refreshing state... [id=igw-0a0b82beebfc6267f]
aws_subnet.my_subnet: Refreshing state... [id=subnet-0d18b566fb2b523a2]
aws_route_table.my_rt: Refreshing state... [id=rtb-06eb0aabaf6c42739]
aws_route_table_association.my_rta: Refreshing state... [id=rtbassoc-08f1a2d3fb30ea496]
```

```
}

Plan: 1 to add, 0 to change, 0 to destroy.
aws_security_group.web_sg: Creating...
aws_security_group.web_sg: Creation complete after 6s [id=sg-0424d0903f780e79a]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\LENOVO\Desktop\Terraform -Final> }
```

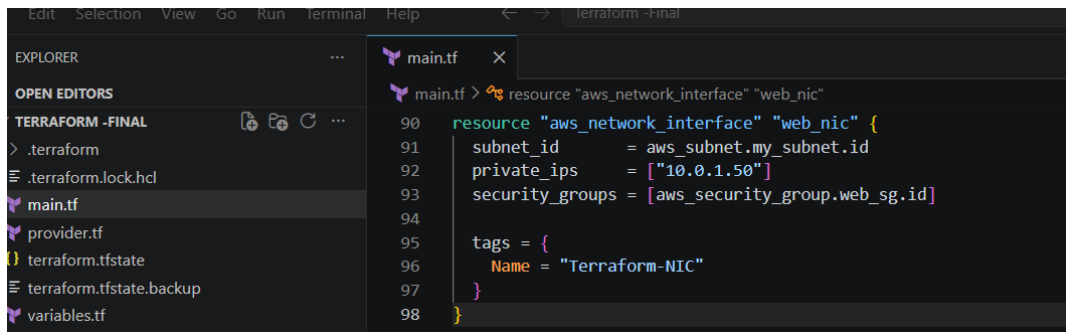
- Now check in VPC Security Groups in Inbound rules ports are added

The screenshot displays the AWS Management Console interface for a VPC Security Group. The left sidebar shows the navigation menu with 'Security groups' highlighted. The main content area shows the details for the security group 'sg-0424d0903f780e79a - Terraform-Web-SG'. The 'Inbound rules' tab is active, showing a table of three inbound rules. The first rule is for HTTPS (port 443), the second is for SSH (port 22), and the third is for HTTP (port 80). The 'Outbound rules' tab shows one rule for allowing all traffic.

Security group rule ID	IP version	Type	Protocol	Port range	Source
sg-0edaebda8af7281d	IPv4	HTTPS	TCP	443	0.0.0.0/0
sg-023476ac6c686aa58	IPv4	SSH	TCP	22	0.0.0.0/0
sg-09e105990a508a69b	IPv4	HTTP	TCP	80	0.0.0.0/0

7) Create a network interface with an Ip in the subnet that was created in step 4

- Created a network interface in main.tf file



```
main.tf
resource "aws_network_interface" "web_nic" {
  subnet_id      = aws_subnet.my_subnet.id
  private_ips    = ["10.0.1.50"]
  security_groups = [aws_security_group.web_sg.id]

  tags = {
    Name = "Terraform-NIC"
  }
}
```

- Now do terraform plan and apply

```
PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform plan
aws_vpc.my_vpc: Refreshing state... [id=vpc-075aae91705e8cdd3]
aws_internet_gateway.my_igw: Refreshing state... [id=igw-0a0b82beebfc6267f]
aws_subnet.my_subnet: Refreshing state... [id=subnet-0d18b566fb2b523a2]
aws_security_group.web_sg: Refreshing state... [id=sg-0424d0903f780e79a]
aws_route_table.my_rt: Refreshing state... [id=rtb-06eb0aabaj6c42739]
aws_route_table_association.my_rta: Refreshing state... [id=rtbassoc-08f1a2d3fb30ea496]

Terraform used the selected providers to generate the following execution plan. Resource
```

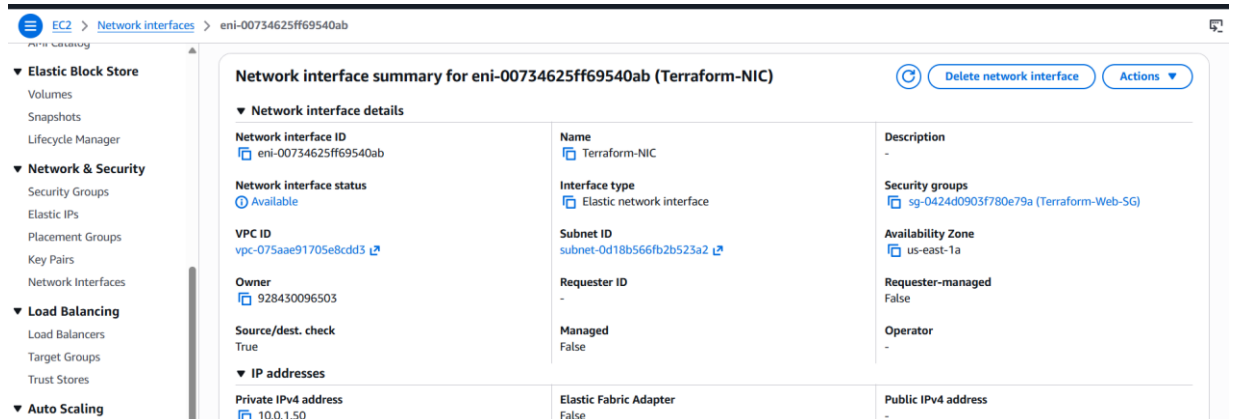
```
PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform apply -auto-approve
aws_vpc.my_vpc: Refreshing state... [id=vpc-075aae91705e8cdd3]
aws_internet_gateway.my_igw: Refreshing state... [id=igw-0a0b82beebfc6267f]
aws_subnet.my_subnet: Refreshing state... [id=subnet-0d18b566fb2b523a2]
aws_security_group.web_sg: Refreshing state... [id=sg-0424d0903f780e79a]
aws_route_table.my_rt: Refreshing state... [id=rtb-06eb0aabaj6c42739]
aws_route_table_association.my_rta: Refreshing state... [id=rtbassoc-08f1a2d3fb30ea496]

Terraform used the selected providers to generate the following execution plan. Resource
```

```
Plan: 1 to add, 0 to change, 0 to destroy.
aws_network_interface.web_nic: Creating...
aws_network_interface.web_nic: Creation complete after 3s [id=eni-00734625ff69540ab]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\LENOVO\Desktop\Terraform -Final>
```

- Now check in EC2 Network Interface created or not



EC2 > Network interfaces > eni-00734625ff69540ab

Network interface summary for eni-00734625ff69540ab (Terraform-NIC)

Network interface details		
Network interface ID eni-00734625ff69540ab	Name Terraform-NIC	Description -
Network interface status Available	Interface type Elastic network interface	Security groups sg-0424d0903f780e79a (Terraform-Web-SG)
VPC ID vpc-075aae91705e8cdd3	Subnet ID subnet-0d18b566fb2b523a2	Availability Zone us-east-1a
Owner 928430096503	Requester ID -	Requester-managed False
Source/dest. check True	Managed False	Operator -
IP addresses		
Private IPv4 address 10.0.1.50	Elastic Fabric Adapter False	Public IPv4 address -

8) Assign an elastic IP to the network interface created in step 7

- Assign Elastic IP to the Network Interface

```
main.tf  X
main.tf > resource "aws_eip" "web_eip"

100 resource "aws_eip" "web_eip" {
101     domain                = "vpc"
102     network_interface      = aws_network_interface.web_nic.id
103     associate_with_private_ip = "10.0.1.50"
104
105     depends_on = [
106         aws_internet_gateway.my_igw
107     ]
108
109     tags = {
110         Name = "Terraform-EIP"
111     }
112 }
```

- Now do terraform commands plan and apply

```
PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform plan
aws_vpc.my_vpc: Refreshing state... [id=vpc-075aae91705e8cdd3]
aws_internet_gateway.my_igw: Refreshing state... [id=igw-0a0b82beebfc6267f]
aws_subnet.my_subnet: Refreshing state... [id=subnet-0d18b566fb2b523a2]
aws_security_group.web_sg: Refreshing state... [id=sg-0424d0903f780e79a]
aws_route_table.my_rt: Refreshing state... [id=rtb-06eb0aabaf6c42739]
aws_network_interface.web_nic: Refreshing state... [id=eni-00734625ff69540ab]
aws_route_table_association.my_rta: Refreshing state... [id=rtbassoc-08f1a2d3fb30ea49]
```

Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exact

```
PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform apply -auto-approve
```

```
aws_vpc.my_vpc: Refreshing state... [id=vpc-075aae91705e8cdd3]
aws_internet_gateway.my_igw: Refreshing state... [id=igw-0a0b82beebfc6267f]
aws_subnet.my_subnet: Refreshing state... [id=subnet-0d18b566fb2b523a2]
aws_security_group.web_sg: Refreshing state... [id=sg-0424d0903f780e79a]
aws_route_table.my_rt: Refreshing state... [id=rtb-06eb0aabaf6c42739]
aws_network_interface.web_nic: Refreshing state... [id=eni-00734625ff69540ab]
aws_route_table_association.my_rta: Refreshing state... [id=rtbassoc-08f1a2d3fb30ea496]
```

Plan: 1 to add, 0 to change, 0 to destroy.

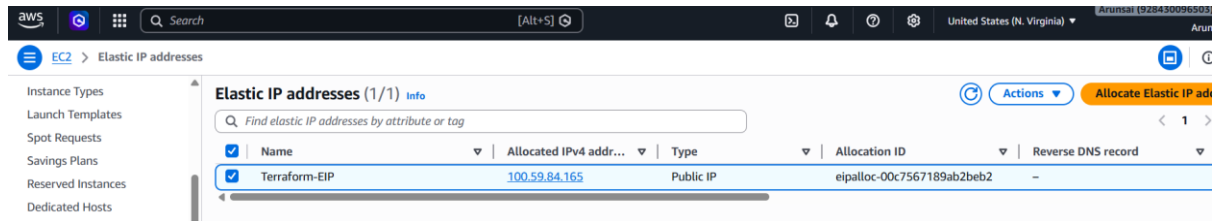
aws_eip.web_eip: Creating...

aws_eip.web_eip: Creation complete after 9s [id=eipalloc-00c7567189ab2beeb2]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

```
PS C:\Users\LENOVO\Desktop\Terraform -Final>
```

- Now check In EC2 Elastic IP



9) Create Ubuntu server and install/enable apache2

- Create Ubuntu server and installed Apache 2

```
main.tf > resource "aws_instance" "web_server"
114 resource "aws_instance" "web_server" {
115     ami           = "ami-0b6d9d3d3ba97d99"
116     instance_type = "t3.micro"
117     key_name      = "aws"
118
119     network_interface {
120         network_interface_id = aws_network_interface.web_nic.id
121         device_index         = 0
122     }
123
124     user_data = <<-EOF
125         #!/bin/bash
126         apt update -y
127         apt install apache2 -y
128         systemctl start apache2
129         systemctl enable apache2
130
131         echo "<h1>Terraform Final Task Completed</h1>" > /var/www/html/index.html
132     EOF
133
134     tags = {
135         Name = "Terraform-Web-Server"
136     }
137 }
138
139 }
```

- Now do terraform commands like plan and apply

```
PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform validate
Success! The configuration is valid.

PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform plan
aws_vpc.my_vpc: Refreshing state... [id=vpc-075aae91705e8cdd3]
aws_internet_gateway.my_igw: Refreshing state... [id=igw-0a0b82beebfc6267f]
aws_subnet.my_subnet: Refreshing state... [id=subnet-0d18b566fb2b523a2]
aws_security_group.web_sg: Refreshing state... [id=sg-0424d0903f780e79a]
aws_route_table.my_rt: Refreshing state... [id=rtb-06eb0aabaf6c42739]
aws_network_interface.web_nic: Refreshing state... [id=eni-00734625ff69540ab]
aws_eip.web_eip: Refreshing state... [id=eipalloc-00c7567189ab2beb2]
aws_route_table_association.my_rta: Refreshing state... [id=rtbassoc-08f1a2d3fb30ea496]
```

```
PS C:\Users\LENOVO\Desktop\Terraform -Final> terraform apply -auto-approve
aws_vpc.my_vpc: Refreshing state... [id=vpc-075aae91705e8cdd3]
aws_internet_gateway.my_igw: Refreshing state... [id=igw-0a0b82beebfc6267f]
aws_subnet.my_subnet: Refreshing state... [id=subnet-0d18b566fb2b523a2]
aws_security_group.web_sg: Refreshing state... [id=sg-0424d0903f780e79a]
aws_route_table.my_rt: Refreshing state... [id=rtb-06eb0aabaf6c42739]
aws_network_interface.web_nic: Refreshing state... [id=eni-00734625ff69540ab]
aws_route_table_association.my_rta: Refreshing state... [id=rtbassoc-08f1a2d3fb30ea496]
aws_eip.web_eip: Refreshing state... [id=eipalloc-00c7567189ab2beb2]
```

(and one more similar warning elsewhere)

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\LENOVO\Desktop\Terraform -Final> 
```

- Now check new EC2 instance created

☰

EC2 > Instances

EC2

Dashboard

AWS Global View

Events

Instances

Instances

Instance Types

Launch Templates

Instances (1/1) Info

Last updated 7 minutes ago

Connect

Instance state

Actions

Save filter sets

Choose filter set

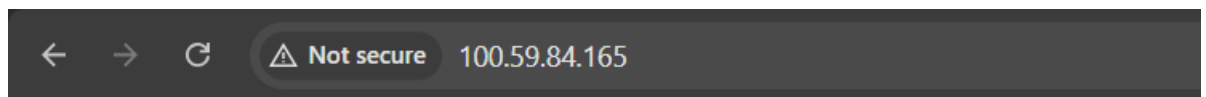
Find Instance by attribute or tag (case-sensitive)

Instance state = running

Clear filters

☒	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Put
☒	Terraform-We...	i-0d68281fc9b43df69	Running	t3.micro	2/2 checks passed	us-east-1a	-

- Now check in browser with Public IP



Terraform Final Task Completed